OGC2026 Technical Report

Abstract. The challenge asks for a joint spatial and temporal plan for shipyard blocks: every block must receive a bay, an orientation, a position, and an entry day so that a weighted sum of total tardiness (Z_1), workload imbalance (Z_2), and bay-preference regret (Z_3) is minimized under exact polygonal, multi-layer, and crane-accessibility feasibility rules. Our solver couples a compiled C++ engine with a defensive Python driver. The engine implements a conservative feasibility oracle that reproduces the official checker's five rules, and runs a four-thread portfolio of adaptive large-neighborhood search (ALNS) annealers whose destroy operators include priced, goal-directed ejection chains gated by dimensionless instance predicates. Cheap atomic moves — slot swaps, in-place shifts, and score-neutral spatial compaction — attack the measured failure mode of dense boards, geometric fragmentation. A memetic population loop with consensus-locking crossover serves tight-budget instances, and the tail of the time limit is split adaptively between annealing restarts and an exact CP-SAT re-placement subsolver made viable by engine-side pair-conflict construction. All schedules are scale-invariant in the block count, every returned solution passes the official checker, and a construction-feasible fallback guarantees a valid answer under any failure. On the public evaluation instances the solver attains objectives of 80,145 to 29.1M at official time limits with zero infeasibilities or overruns, and it remains feasible on all 40 final-round training instances.

1 Introduction

1.1 Challenge Problem & Main Difficulties

Each block i carries a release day R_i, a processing time P_i, a due day D_i, a workload L_i, bay preference scores S_{ij}, and a shape given by K_i stacked polygonal layers. A solution assigns to every block a bay, one of its allowed orientations, an integer position (x, y), and an entry day e_i; the block exits at $f_i = e_i + P_i$. The objective is $w_1 Z_1 + w_2 Z_2 + w_3 Z_3$, where Z_1 is total tardiness $\sum_i \max(0, f_i - D_i)$, Z_2 is the maximum area-normalized workload imbalance over bays and days, and Z_3 is the preference regret $\sum_i (S_i^{\max} - S_{i,j(i)})$.

Three properties make the problem hard. First, feasibility is genuinely geometric: blocks are non-convex polygons stacked in layers, two blocks conflict only when time-overlapping same-layer polygons intersect with positive area, and a crane rule requires the moving block's layer to clear *all* layers at or above its settling level along a vertical descent, at both entry and exit. Bounding-box reasoning is unsound in both directions: it forbids legal interlocked placements and cannot express the asymmetric crane relation. Second, the objective mixes two regimes. On some instances tardiness dominates by an order of magnitude; on others tardiness is zero and the game is pure preference regret, so any fixed

tuning for one regime fails the other. Third, evaluation instances are hidden: constants tuned to visible sizes silently change the algorithm's behavior at other scales, and every run must return a feasible solution within a hard time limit.

1.2 Contributions

1. A compiled, conservative feasibility oracle reproducing the five rules of the official checker (same-layer overlap, crane entry, crane exit, and non-blocking of later entries and exits), audited against the checker on uniform, contact-tie, and nesting cases with zero discrepancies.
2. A four-thread portfolio ALNS with ten adaptive destroy operators, including two *priced* directed ejection operators (a preference-chain and a tardiness min-cut ejection) whose activation is gated by dimensionless predicates computed from the instance's own weights, not tuned thresholds.
3. Atomic moves that require no free space — tardy-initiated slot swaps, in-place shifts, and provably score-neutral spatial compaction — derived from a measured diagnosis that residual tardiness is caused by geometric fragmentation rather than lack of capacity.
4. An adaptive tail allocator that splits the final fraction of the budget between annealing restarts and an *exact* CP-SAT re-placement subsolver; engine-side pair-conflict construction accelerates the conflict model by orders of magnitude and is what makes the exact arm affordable in the limit.
5. Scale-invariant search schedules (cooling, reheating, removal sizes, stagnation windows all relative to n) and a defensive driver in which every returned solution passes the official checker, with a construction-feasible fallback for worst-case degradation.

2 Algorithm Description

2.1 Overall Algorithm Framework

The driver calls a single compiled engine through the following pipeline:

1. **Seed construction.** Six insertion orders (earliest-due-date, slack-based, noised variants) are replayed through the engine's greedy placer; the best three distinct layouts become seeds. The placer performs earliest-entry first-fit in three phases per block: a full corner scan at $e = R_i$, a local scan around each residual block's exit event ascending in time (space opens only when someone leaves), and a sparse dense sweep over the remaining days as a completeness net.
2. **Portfolio search.** Four OpenMP threads anneal in parallel with heterogeneous personas: removal-size multipliers, initial temperatures, pure-annealing versus hill-climbing tails, operator subsets, and area-screen regimes all differ, and the four starting points hedge between the best seed, a fresh construction, and the runners-up. Acceptance is simulated annealing on the exact objective; half of the personas switch to hill climbing in the last quarter of their budget.

3. **Population path.** When the per-block budget is tight (sec/$n \leq 1.6$ and sec ≥ 150), a memetic outer loop replaces the flat portfolio: distinct constructions are hardened by short bursts into a founder population, and offspring are produced by consensus-locking crossover — only blocks whose bay assignment both elite parents agree on are locked, the rest is re-annealed — replacing the worst member.
4. **Slice-refined tail.** The last 30% of the budget is cut into slices; each slice is given to whichever of two arms showed the higher measured improvement rate per second in the previous slice: (a) portfolio annealing restarted from the global best, or (b) an exact re-placement polish that frees a random subset of ≈ 25 blocks seeded by a tardy block and re-places them *optimally* against the fixed remainder with CP-SAT. The tail is skipped when the population path is armed, and it always stops before the checker gate's repair reserve.
5. **Assignment descent and gate.** A final descent accepts only exactly-priced improving reassignments (with the assignment fixed, Z_2 and Z_3 are closed-form and the checker rules close within bays). Every candidate then passes the official checker; if repair cost exceeds a small bound, runner-up candidates are auditioned through the same gate, and a construction-feasible trivial solution guarantees a valid return in all cases.

Feasibility during search is decided by the engine's conservative oracle, which evaluates the checker's five rules for a candidate block against the residents of its bay. Polygon overlap follows the checker's semantics — proper edge intersection or strict-interior containment, with edge and vertex contact allowed — and the crane rules test the vertical descent of the moving layer against all resident layers at or above the settling level, at entry and at exit, including non-blocking of residents that enter or exit during the block's stay. Same-day operations are ordered exits-first with last-in-first-out exits, exactly as the checker replays them. Bounding boxes and a bit-grid occupancy prefilter are used only as short-circuit *gates* in front of the exact tests, never as the decision.

2.2 Key Ideas that Succeeded

Exact geometry as the oracle. On dense boards the peak bay occupancy is 103–105% of bay area by bounding boxes but only 58–65% by true polygons: roughly a third of the usable capacity exists only through interlocking, and a bounding-box oracle would forbid it. The oracle was audited against the official checker on 2,000 uniform samples, 90,000 contact-tie cases, and 1,500 nesting cases with zero discrepancies.

Priced, gated ejection operators. Uniform random destruction cannot compose the moves that matter on saturated boards. The preference-chain operator empties a preferred bay only while the ejection cost stays below the preference gain, then forces the mover in at its on-time entry $e^* = \max(R_i, D_i - P_i)$; the tardiness min-cut operator scans *all* bays for the hole whose ejection set is cheapest inside a tardy block's on-time window and removes exactly that set.

Both are gated by dimensionless predicates ($w_3 Z_3 > w_1 Z_1$; zero tardiness with small regret) recomputed during search from the instance's own weights, so the gating transfers to hidden instances by construction.

Atomic moves against fragmentation. Instrumented runs show the repair step never fails to place a block (0 failures in over 10^6 iterations per run); tardiness arises because the first feasible day is late. Diagnostics attribute this to fragmentation: free area exists but is scattered inside other blocks' bounding boxes, and no tardy block has any feasible position in any bay during its release window. Three moves that need no free space follow: slot swaps between a tardy block and a resident (2% of iterations), in-place shifts within a bay at fixed entry (10%), and end-of-round spatial compaction that pushes every resident to its leftmost legal x. Since position does not enter the objective, compaction is score-neutral by construction and purely manufactures contiguous space; it is paired with a time compaction that only advances entries.

Exact subsolver in the tail. Re-placing small congested neighborhoods exactly proves the boards locally optimal (gap 0.0%), but re-placing *random* tardy-seeded subsets does improve them. The decisive step was moving conflict construction into the engine: candidate-pair conflicts (same-layer settling overlap plus vertical crane sweeps) for ∼1,300 candidate placements — about 10^5 pairs — are built in under 0.1 s, several hundred times faster than the interpreted equivalent, which flips the cost-benefit sign of the exact arm inside the time limit. Every CP-SAT proposal is re-verified by the engine's own oracle before acceptance. Disabling the slice-refined tail worsens 26 of 40 final-round training instances (median +1.79%), with the largest losses on the largest boards.

Scale invariance. Every schedule constant is expressed relative to n: the cooling factor $\exp(-0.06/n)$ per iteration, reheating after $6n$ stagnant iterations, small removals of up to $\max(8, 0.03n)$ blocks, large removals of $0.05n$ to $0.15n$ blocks, return to the incumbent after $20n$ stagnant iterations, and a $1/n$-weighted slack surrogate. With absolute constants the same code becomes a different algorithm at unseen sizes.

Measurement discipline. Per-seed dispersion of 5–8% (occasionally 25%) makes single-run comparisons meaningless. Every adoption decision used paired comparisons — the same instances and seeds for both configurations — with a sign test over the 40 final-round training instances, and refactorings were accepted only on byte-identical outputs at fixed iteration counts. This discipline reversed one earlier adoption (see below).

2.3 Key Ideas that Failed

Heterogeneous acceptance. Annealing half the portfolio with late acceptance hill climbing looked profitable on six public instances but was reversed by the 40-instance paired sign test (26 losses to 14 wins, $p = 0.04$, mean +2.34%); it was retracted. *Smarter per-iteration placement.* Best-of-R repair, contact-score best-fit, and raster-NFP tight packing all bought per-iteration quality at a larger cost in iteration count and lost 1.5–8.4% end to end; throughput governs quality. *Speed without throughput.* Two time-keyed candidate indexes cut loop visits

by 55–75% and were byte-identical to the reference, yet gained $\leq 3\%$ and were rejected. *Exact decomposition masters.* A set-partitioning master over bay-plan columns (and a block-ownership relaxation of it) closed to proven optimality with 0.0% improvement — bay-plan recombination has no slack. A global placement MIP with tardiness conflict cuts improved one preference-dominated instance marginally but did not scale. *Vacuous lower bound.* An area-relaxation bound closes to $Z_1 = 0$ on all four public instances, yet its certificate assignment is not geometrically realizable; the true optimality gap remains unknown. *Targeted room-making.* Forcing a mover into its preferred bay on time and exactly re-placing the displaced neighbors works mechanically but never paid for the displacement.

2.4 Further Improvement Plan

The measured residual gap sits in globally different geometric layouts, not in local re-placement or scheduling. Three directions follow. (i) Replace the quadratic pairwise conflict model with an exact sub-quadratic formulation so the exact replacement arm can scale from ≈ 25 to 50 or more freed blocks. (ii) Recover the $\approx 14\%$ cross-thread final-value dispersion of the portfolio with a temperature-preserving migration design; naive reallocation schemes measurably fail. (iii) Density-gated polygon-contact candidate generation, which wins on crowded boards and loses on sparse ones, pending validation on enough boards to trust the gate on hidden instances.

3 Implementation Details

The submission consists of a Python driver (`myalgorithm.py`, roughly one thousand lines), a C++17 engine of roughly four thousand lines compiled as a single translation unit (`src/api.cpp` including geometry, crane, core, occupancy, repair, and search headers) into a pybind11 module, the organizer-provided checker utilities (unmodified), and a build note with the exact compile command and toolchain. The engine is built with `g++ -O3` (C++17, OpenMP, `x86-64-v3`, `-ffp-contract=off`, LTO) for CPython 3.12, and a prebuilt binary for the evaluation environment is included. Beyond the compiled extension, the driver needs only the Python standard library; CP-SAT is used through OR-Tools only when the interpreter already provides it.

The driver is defensive by design: the engine, the CP-SAT dependency, and the checker are imported optionally, and every degradation path is closed — if CP-SAT is unavailable the exact arm silently drops and the annealing arm keeps the tail; in the worst case a pure-Python, construction-feasible trivial solution is returned. The entry point never raises. Threads are capped at $\min(4, \text{cpu_count})$; no environment configuration is read, so the evaluation environment exercises exactly the shipped defaults. Peak memory stays below 600 MB on the largest boards.

Table 1. Single fixed-seed runs of the submitted solver at official time limits on the public evaluation instances. Z_1 in block-days; weights (w_1, w_2, w_3) are part of each instance.

Inst.	n	m	TL (s)	Objective	Z_1	Z_2	Z_3	Wall (s)
P3	200	3	240	80,145	0	3,399	421	232.2
P4	150	3	480	3,031,533	178	3,287	4,235	470.7
P5	200	4	600	10,415,006	735	683	4,590	587.3
P6	250	3	900	29,145,539	4,209	1,042	7,172	887.8

Determinism: within an iteration-governed round, the search trajectory is a function of the seed and round count only; byte-identical results were verified across nodes. Faster hardware packs more rounds into the limit, and the best value is monotone in rounds, which is the basis for transferring locally measured results to the evaluation server. Deadline safety is enforced end to end: the tail stops before the checker gate's repair reserve, and the gate itself has a bounded rescue ladder with runner-up audition.

4 Computational Results

Table 1 reports the submitted solver on the four public evaluation instances at official time limits, as a single fixed-seed run of the exact submitted code — the same condition under which the solver is scored. All runs are feasible under the official checker, with no time-limit overruns. The three metrics are reported raw; the objective equals $w_1 Z_1 + w_2 Z_2 + w_3 Z_3$ with the instance weights shown.

The decomposition shows the two regimes explicitly: P3 is a pure assignment game ($Z_1 = 0$; the objective is dominated by preference regret), while on P4–P6 weighted tardiness contributes 76–97% of the objective. The dimensionless gates of Section 2 detect this split at run time; no instance-specific configuration exists in the code.

On the 40 final-round training instances ($n \leq 300$, $m \leq 5$, weight ratios w_1/w_3 spanning 4–107), the solver is feasible on all instances with zero checker-gate failures and zero overruns, and the portfolio sustains roughly 0.8–1.3×10^6 search iterations per run at official limits. Under extended budgets the engine continues to convert time into quality (8 of 10 large boards improve from 600 s to 1,500 s, up to -12.6%), indicating that the search does not freeze within the official limits.

Honest limitations: the residual gap to the strongest known values on the public instances is concentrated in tardiness. Local analysis proves our boards exactly optimal for any re-placement of the 20–30 most congested blocks and time-globally optimal at fixed geometry, so the remaining gap lives in globally different layouts that our operators do not reach; the area-relaxation lower bound is vacuous, so the distance to true optimality is unknown.

5 Team Members' Contributions (Not Applicable to Single-Member Teams)

This is a single-member team. Problem analysis, algorithm design, mathematical modeling, implementation, experimentation, validation, and report preparation were carried out solely by the participant.

6 AI Use Declaration

Large-language-model coding assistants (Claude-family models used through an agentic coding tool) were used throughout the project in three roles. *Code:* the assistant drafted and refactored large parts of the C++ engine and the Python driver under the participant's direction; all generated code was reviewed by the participant and validated mechanically — byte-identity checks for refactorings, fixed-seed canaries, unit and battery tests, and the official feasibility checker on every produced solution. *Experiments:* the assistant helped build the evaluation harnesses, scheduled runs, and summarized measurements; experimental design, acceptance criteria (paired sign tests at official time limits), and all adopt-or-reject decisions were made by the participant, and several assistant-proposed changes were rejected by these tests. *Report:* the assistant helped draft and edit this report from the project's measurement records; the participant verified every quantitative claim against the frozen logs and the submitted code and takes full responsibility for the content. No AI system had autonomous decision authority over the submitted algorithm or this report.

References

1. Ropke, S., Pisinger, D.: An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. Transportation Science 40(4), 455–472 (2006)
2. Shaw, P.: Using constraint programming and local search methods to solve vehicle routing problems. In: Principles and Practice of Constraint Programming (CP 1998), LNCS 1520, pp. 417–431. Springer (1998)
3. Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P.: Optimization by simulated annealing. Science 220(4598), 671–680 (1983)
4. Perron, L., Furnon, V.: OR-Tools CP-SAT solver, version 9. Google (2024)